

molsim-agent

A transparent scientific agent for molecular simulation

Open-source research project

September 25, 2026

Project purpose

- ▶ The LLM interprets a scientific objective and chooses the next action.
- ▶ Trusted Python and ASE tools perform parsing, simulation, analysis, and validation.
- ▶ The runtime records observations, assumptions, artifacts, and provenance.
- ▶ Structure conversion is one workflow among several.

What makes this an agent?

1. Receive an objective and current state.
2. Expose a bounded set of typed tools.
3. Let the model request a tool or finish.
4. Execute the request after validation.
5. Return the observation to the model.
6. Stop after a validated result, failure, or iteration limit.

The explicit loop

User objective → LLM planner → validated plan

Tool call → Python tool → observation

Observation → state update → LLM

Final scientific report

The runtime never treats plausible text as evidence. A successful tool observation is the boundary between a plan and an executed calculation.

LLM planner for flexible requests

- ▶ Receives the complete objective, workspace files, and registered tool schemas.
- ▶ Decomposes multi-task requests into ordered steps.
- ▶ Produces structured arguments such as temperature, duration, ensemble, and source.
- ▶ The runtime validates every step before execution.
- ▶ If planning fails, the explicit tool-calling loop remains the safe fallback.

No prompt-specific branch

New user wording is handled by the planner rather than by adding a new hard-coded exception to the central runtime.

Scientific orchestrator

- ▶ Understand the requested quantity, code, method, and constraints.
- ▶ Decide which workflow and tools are needed.
- ▶ Assess capabilities before claiming execution.
- ▶ Identify defaults, derived parameters, missing inputs, and risks.
- ▶ Interpret deterministic results without fabricating numbers.

Capability assessment

Structured status

available, needs_implementation, needs_dependency, needs_external_data, needs_compute, needs_user_input, unsupported

Example

The agent understands an orientational autocorrelation request, but no trusted tool is registered. It can propose implementation with NumPy/MDAnalysis instead of pretending that the observable was computed.

Workflow registry

Workflow	Tool	Responsibility
Conversion	<code>convert_structure</code>	ASE I/O and validation
Simulation	<code>run_md</code>	Bounded ASE protocol
Analysis	<code>mean_squared_displacement</code>	Deterministic observable
Template	<code>prepare_*_inputs</code>	Defaults plus missing inputs

The orchestrator uses a registry contract instead of a growing code-specific branch in the central loop.

Scientific plans and parameter provenance

Every generated protocol can label each parameter as:

- ▶ `user`: supplied explicitly;
- ▶ `derived`: calculated from the request;
- ▶ `default`: standard starting assumption;
- ▶ `missing`: requires evidence or user input.

This makes a template inspectable before it becomes a production calculation.

Example: VASP AIMD

- ▶ Generated: INCAR and Gamma-point KPOINTS.
- ▶ Derived: $1 \text{ ps} / 1 \text{ fs} = 1000 \text{ steps}$.
- ▶ Default: fixed cell and standard MD mode.
- ▶ Missing: POTCAR, functional, ENCUT, spin, smearing, convergence, executable.

Safety rule

The agent writes a reviewable template, not a claim that the production setup is valid.

Example: LAMMPS MD

- ▶ Generated: deterministic structure data and `in.molsim`.
- ▶ Default: metal units, 1 fs timestep, 1000 steps, NVT protocol.
- ▶ Explicit placeholders: `pair_style` and `pair_coeff`.
- ▶ Missing: force field or ML potential, masses, validated coefficients, executable.

The POSCAR does not contain enough information to choose a classical potential.

Deterministic scientific tools

- ▶ ASE reads and writes structures.
- ▶ ASE calculators execute single-point, optimization, and bounded MD workflows.
- ▶ Analysis tools compute trajectory summaries, distances, and MSD.
- ▶ Validation compares atom count, symbols, positions, cell, PBC, and optional data.
- ▶ Experiment records store hashes, parameters, outputs, warnings, and versions.

Example: GROMACS MD

- ▶ Generated in the workspace output directory: ASE-based `.gro`, generic `md.mdp`, and an explicitly incomplete topology template.
- ▶ Defaults and derived values are reported before production use.
- ▶ Missing: force field, atom types, charges, bonded parameters, topology, and executable configuration.

Scientific boundary

The agent never invents a chemically valid GROMACS topology from a POSCAR alone.

Safety boundaries

- ▶ All paths stay inside one workspace.
- ▶ Existing files are protected by default.
- ▶ No arbitrary shell execution.
- ▶ Generated code stays outside trusted `src/`.
- ▶ A tool failure is an observation, not a successful result.
- ▶ Unsupported scientific mappings are reported explicitly.

Small-model and full profiles

- ▶ `local`: compact deterministic routing for Ollama and small models.
- ▶ `scientific`: full tool registry and LLM-assisted planning.
- ▶ `debug`: detailed requests, tool calls, observations, and timing.

Typical commands

```
molsim-agent -w ./simulation -m qwen3:8b
```

```
molsim-agent -w ./simulation -m qwen3:8b -preset scientific
```

Adding a new scientific code

1. Define a structured specification.
2. Register a workflow and its deterministic tool.
3. Declare dependencies, cost, risk, and determinism.
4. Label defaults and missing inputs.
5. Add tests with a lightweight calculator or test double.
6. Update the skill and this presentation.

Current status and roadmap

Implemented: explicit loop, registry, capabilities, ASE simulation tools, templates, analysis, provenance, safety, Ollama and hosted backends.

Next: literature/documentation retrieval, RDF, stronger generated-code isolation, MACE/UMA execution, comparative experiments, and additional simulation codes.

Key design principle

The LLM decides what should happen.

Trusted scientific software decides how it happens.

Validation decides whether the result can be trusted.